
Chapter 1: Getting started with redis

Remarks

This section provides an overview of what Redis is, and why a developer might want to use it.

It should also mention any large subjects within Redis, and link out to the related topics. Since the documentation for Redis is new, you may need to create initial versions of those related topics.

Versions

Version	Release Date
3.2.3	2016-08-02
3.2.2	2016-07-28

Examples

Overview

Redis is an in-memory remote database that offers high performance, replication, and a unique data model to produce a platform for solving problems. Redis is an open source (BSD licensed), in-memory data structure , used as database, cache and message broker. It is categorized as a NoSQL key-value store. It supports data structures such as strings, hashes, lists, sets, sorted sets with range queries, bitmaps, hyperloglogs and geospatial indexes with radius queries. Supporting five different types of data structures,

1. STRING (Operate on the whole string, parts, integers and floats)
2. LIST (Push or pop items from both ends)
3. SET (Add, fetch, remove, check, intersect, union, difference etc)
4. HASH (store, fetch, remove in hash)
5. ZSET (same as set but in ordered way)
6. GEO (Add, update, delete latitude and longitude, get within given radius)

Redis has built-in replication, Lua scripting, LRU eviction, transactions and different levels of on-disk persistence(sync/async).

Prior to version 3, Redis works in master-slave mode and required Redis-Sentinel to provide high-availability. Only master accepts writes and syncs data to its slaves by forking.

From version 3, Redis works & recommends multi-master mode where failover, sharding/partitioning, resharding features are in-built. Redis-Sentinel is not required from version-3. In order for the redis cluster to operate a minimum of 3 master nodes/processes are required.

Additional features are replication, persistence, and client-side sharding. Redis accommodates a wide variety of problems that can be naturally mapped into what Redis offers, allowing you to solve your problems without having to perform the conceptual work required by other databases.

Redis command line interface

`redis-cli` is the Redis command line interface program that allows to send commands to Redis and read the replies sent by the server, directly from the terminal. Basic command line usage is below:

Access to redis:

```
$ redis-cli
127.0.0.1:6379>
```

Access to redis with authentication:

```
$ redis-cli -a myPassword
127.0.0.1:6379>
```

Select database and show database size (default database number is 0):

```
127.0.0.1:6379> dbsize
(integer) 2
127.0.0.1:6379> select 1
OK
127.0.0.1:6379[1]> dbsize
(integer) 20
```

Get information and statistics about the server:

```
127.0.0.1:6379> info
redis_version:2.4.10
redis_git_sha1:00000000
redis_git_dirty:0
arch_bits:64
multiplexing_api:epoll
gcc_version:4.4.6
process_id:947
uptime_in_seconds:873394
uptime_in_days:10
lru_clock:118108
used_cpu_sys:19.55
used_cpu_user:397.46
used_cpu_sys_children:0.00
used_cpu_user_children:0.00
connected_clients:1
connected_slaves:0
client_longest_output_list:0
client_biggest_input_buf:0
blocked_clients:0
used_memory:14295792
used_memory_human:13.63M
used_memory_rss:19853312
```

```
used_memory_peak:14295760
used_memory_peak_human:13.63M
mem_fragmentation_ratio:1.39
mem_allocator:jemalloc-2.2.5
loading:0
aof_enabled:0
changes_since_last_save:0
bgsave_in_progress:0
last_save_time:1468314087
bgrewriteaof_in_progress:0
total_connections_received:2
total_commands_processed:2
expired_keys:0
evicted_keys:0
keyspace_hits:0
keyspace_misses:0
pubsub_channels:0
pubsub_patterns:0
latest_fork_usec:0
vm_enabled:0
role:master
db0:keys=2,expires=0
db1:keys=20,expires=0
```

Exiting from the redis-cli:

```
127.0.0.1:6379> exit
```

Redis "Hello World"

First you need to install and start your Redis server, check the link below that can help you to install redis on you server or local machine.

Installation and Setup

Now open your command prompt and run command `redis-cli` :

To save first set `>SET 'keyname' then 'value'`

```
127.0.0.1:6379> SET hkey "Hello World!"
```

Press Enter you should see

```
OK
```

Then enter:

```
GET hkey
```

you should see:

```
"Hello World!"
```

Screen output example:

```
127.0.0.1:6379> SET hkey "Hello World!"
OK
127.0.0.1:6379> GET hkey
"Hello World!"
127.0.0.1:6379>
```

Install Redis by using Docker

It is simple to start using Redis using docker:

```
docker pull redis
docker run -p 6379:6379 --rm --name redis redis
```

Now you have running instance on port 6397

Attention: All data will be deleted, when Redis will be stopped.

To connect the redis-cli, start another docker:

```
docker run -it --link redis:redis --rm redis redis-cli -h redis -p 6379
```

Now you can play around with your redis docker.

Redis installtion on Windows, with Node.js example

Redis has a Windows port provided by 'Microsoft Open Technologies'. You can use the msi installer found on: <https://github.com/MSOpenTech/redis/releases>

After installation completes you can see 'Redis' is a Windows service (and it's status should be "Started")

To write an 'Hello world' example that uses Redis in Node.js (in windows as well) you can use the following npm module : <https://www.npmjs.com/package/redis>

code sample:

```
var redis = require('redis'),
    client = redis.createClient();

client.set('mykey', 'Hello World');
client.get('mykey', function(err,res){
  console.log(res);
});
```

Read Getting started with redis online: <https://riptutorial.com/redis/topic/1724/getting-started-with-redis>

Chapter 2: Backup

Introduction

Backing up a remote Redis instance can be achieved with replication. This is useful if you want to take a snapshot of a dataset prior to upgrading, deleting or changing a Redis database.

Examples

Backup of a remote Redis instance to a local instance

On the machine where you'd like to make the backup, jump to the Redis CLI:

```
redis-cli
```

Password?

If your master Redis DB (the one you want to replicate) has a password:

```
config set masterauth <password>
```

Start replication

Run the following to begin replication:

```
SLAVEOF <host> <port>
```

To check the replication is underway run:

```
INFO replication
```

And you should see output like this:

```
# Replication
role:slave
master_host:some-host.compute-1.amazonaws.com
master_port:6519
master_link_status:up
master_last_io_seconds_ago:3
master_sync_in_progress:0
slave_repl_offset:35492914
slave_priority:100
slave_read_only:1
connected_slaves:0
master_repl_offset:0
repl_backlog_active:0
```

```
repl_backlog_size:1048576
repl_backlog_first_byte_offset:0
repl_backlog_histlen:0
```

Note the `master_link_status` should be `up`.

Checking sync progress

When the sync is complete, the `INFO replication` should show:

```
master_sync_in_progress:0
```

To check the dataset has been synced you could compare the size of the database:

```
DBSIZE
```

Saving a data dump to disk

To save the DB to disk asynchronously:

```
BGSAVE
CONFIG GET dir
```

Then you should find a `dump.rdb` file in the directory listed by the config command.

Halting replication

You can stop replication with:

```
SLAVEOF NO ONE
```

Reference: [Redis replication guide](#)

Read Backup online: <https://riptutorial.com/redis/topic/9369/backup>